



Before Your First Class – Environment Setup

Complete everything on this page before the first lecture.

Lecture 1 assumes all tools are installed and working. We will use many of these tools during the first lecture and all of them throughout the semester.

1. Git

Check if Git is already installed:

```
git --version
```

If you see a version number (e.g., `git version 2.x.x`), skip to **Configure your identity** below.

Install Git:

Platform	Command / Installer
macOS	<code>xcode-select --install</code> (installs Xcode Command Line Tools, which includes Git)
Windows	Download and run the installer at https://git-scm.com/download/win – accept all defaults
Linux (Ubuntu/Debian)	<code>sudo apt install git</code>
Linux (Fedora)	<code>sudo dnf install git</code>

Configure your identity (required before you can make commits):

```
git config --global user.name "Your Name"
git config --global user.email "yournetid@wisc.edu"
```

Use your UW–Madison email so your commits are tied to your university identity on GitLab.

2. VS Code

Download and install Visual Studio Code from <https://code.visualstudio.com/>.

Accept all defaults during installation.

macOS only: Install the `code` shell command so you can open VS Code from the terminal:

1. Open VS Code
2. Press `Cmd+Shift+P` to open the Command Palette
3. Type **"Install 'code' command in PATH"** and press Enter

Verify: `code --version` should print a version number.

3. Claude Code

Claude Code is the AI coding assistant used throughout the course. You are required to have access to Claude Code at least at the Pro level for the entire semester. Check out Anthropic's plans [here](#).

Install the VS Code extension:

1. Open VS Code
2. Press `Ctrl+Shift+X` (Windows/Linux) or `Cmd+Shift+X` (macOS) to open the Extensions panel
3. Search for **Claude Code**
4. Install the extension published by **Anthropic**

Authenticate: Claude Code requires a Claude.ai account. You are required to have a Claude.ai subscription at least at the **Pro** level for the duration of the course. Check out their [pricing](#) and purchase a plan for the semester before the first day of class.

Verify: After authenticating, open any folder in VS Code. Press `Ctrl+Shift+P` / `Cmd+Shift+P` and type "Claude". You should see Claude Code commands in the palette.

Note: If the Claude icon is not showing up after you have opened a folder, you may need to [trust](#) the workspace.

4. Node.js 20

Check if Node.js 20 is already installed:

```
node --version
```

If the output starts with `v20`, you're done.

Install – macOS / Linux (recommended: use nvm):

nvm (Node Version Manager) lets you install and switch between Node versions. This is the cleanest approach on macOS and Linux.

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

Close and reopen your terminal, then:

```
nvm install 20
nvm use 20
nvm alias default 20
```

Install – Windows:

Go to <https://nodejs.org/en/download>. The page now defaults to a newer version, so use the version dropdown to select the latest **20.x** release before downloading, then run the Windows installer and accept all defaults.

Verify:

```
node --version    # v20.x.x
npm --version     # 10.x.x or higher
```

5. Python 3.12

Check what version you have:

```
python3 --version
```

If the output is `Python 3.12.x`, you're done.

Install:

Platform	Steps
macOS	The main download button at https://www.python.org/downloads/ now defaults to a newer version — instead, go to https://www.python.org/downloads/latest/python3.12/ and download the macOS installer, then run it. Or: <code>brew install python@3.12</code> if you use Homebrew.
Windows	The main download button at https://www.python.org/downloads/ now defaults to a newer version — instead, go to https://www.python.org/downloads/latest/python3.12/ and download the Windows installer. Check "Add Python to PATH" on the first screen of the installer — this is easy to miss.
Linux (Ubuntu 22.04+)	<code>sudo apt install python3.12 python3.12-venv python3-pip</code>

Verify:

```
python3 --version    # Python 3.12.x
pip3 --version       # pip 24.x or similar
```

6. Docker Desktop

Docker is used to run multi-service application stacks. You will need it starting in Week 4, but install it now so there's no scramble later.

Download Docker Desktop for your platform from <https://www.docker.com/products/docker-desktop/>.

Platform notes:

- **macOS:** Requires macOS 12 (Monterey) or later. After installation, launch Docker Desktop from Applications and wait for the whale icon in the menu bar to become solid — it takes about a minute to start.
- **Windows:** Requires Windows 10 or 11 with WSL 2. The installer will guide you through enabling WSL 2 if it isn't already active.
- **Linux:** Install Docker Engine and the Docker Compose plugin directly (not Docker Desktop):
<https://docs.docker.com/engine/install/>

Verify (Docker Desktop must be running):

```
docker --version      # Docker version 25.x or later
docker compose version # Docker Compose version 2.x
```

If `docker compose version` fails with "unknown command", try `docker-compose version` (older syntax). If that also fails, Docker is not running — open Docker Desktop first.

7. UW GitLab Account

All course repositories are hosted on the UW–Madison GitLab instance at git.doit.wisc.edu.

Sign in:

1. Go to <https://git.doit.wisc.edu>
2. Click **Sign in with UW–Madison NetID**
3. Authenticate with your NetID and password

Your account is provisioned automatically on first login.

Set up HTTPS authentication with a personal access token so you can clone, push, and pull from anywhere:

Why not SSH? DoIT's GitLab only allows direct SSH connections from an **on-campus IP address** (or over WiscVPN) — see [DoIT's connection docs](#). Since you'll need to push/pull from home, dorms, and elsewhere off-campus, this course uses HTTPS with a personal access token instead, which works over the regular HTTPS port from anywhere, no VPN required.

1. Create a personal access token:

- Click your avatar in the top-right corner of GitLab → **Edit Profile**
- In the left sidebar, click **Access -> Personal Access Tokens**
- Click "Add new token" and give it a name (e.g., "My Laptop") and leave the expiration date at the default, or set one that covers the semester
- Under **Scopes**, check **api**
- Click **Generate token**
- **Copy the token now and save it somewhere safe** (e.g., a password manager) — GitLab only shows it once.

2. Set up a git credential helper so you don't have to paste the token on every push/pull:

◦ **macOS:**

```
git config --global credential.helper osxkeychain
```

◦ **Windows (Git Bash):**

```
git config --global credential.helper manager
```

◦ **Linux:**

```
git config --global credential.helper store
```

(This saves credentials in plain text at `~/.git-credentials`. That's fine for a personal laptop with a semester-scoped token.)

The first time you clone, push, or pull, Git will prompt for a username and password:

- **Username:** your NetID
- **Password:** paste your personal access token (**not** your NetID password)

After that, the credential helper remembers it and you won't be prompted again.

Verify:

```
git ls-remote https://git.doit.wisc.edu/cdis/cs/courses/aicoding/fa26/student-repos/lecture/lectures-  
<yournetid>.git
```

Expected response: a list of refs (branch names and commit hashes), not an authentication error. You'll be prompted for your NetID and token the first time, as described above.

Always on campus, or already using WiscVPN? SSH keys still work as an alternative — see the [DoIT docs](#) for setup. This README standardizes on HTTPS so the steps work for everyone regardless of location.

8. Clone Your Repos

Your repositories have already been created and you've been added as a Developer. You don't need to create anything on GitLab — just clone what's there. You should have been invited (check your email) to your own fork of the lectures repository (`lectures-<yournetid>`) and to a side-scroller project 1 repository.

Find the HTTPS clone URL:

1. Go to your `lectures-<yournetid>` repo on GitLab: <https://git.doit.wisc.edu/cdis/cs/courses/aicoding/fa26/student-repos/lecture/lectures-<yournetid>>
2. Click the blue **Clone** button near the top right of the page
3. Under **Clone with HTTPS**, copy the URL (it starts with [https://git.doit.wisc.edu/...](https://git.doit.wisc.edu/))

Clone it:

```
git clone https://git.doit.wisc.edu/cdis/cs/courses/aicoding/fa26/student-repos/lecture/lectures-<yournetid>.git
```

The first time, Git prompts for your NetID (username) and personal access token (password) — your credential helper remembers it after that. This creates a `lectures-<yournetid>/` directory on your machine with the full repo inside. Git automatically sets `origin` to point back to the GitLab URL you cloned from.

Verify it worked:

```
cd lectures-<yournetid>
git remote -v
```

You should see `origin` listed twice — once for fetch and once for push — both pointing to your GitLab HTTPS URL.

Then clone your P1 repo the same way. You've also been invited to a Project 1 starter repo. Find it on GitLab, copy its HTTPS clone URL, and run `git clone <url>`. Both this repo and your P1 repo will be used during Lecture 1.

This same procedure — find the repo on GitLab, copy the HTTPS clone URL, run `git clone` — is how you'll get every repo this semester onto your machine.

Pre-Class Reading

Complete before coming to the first lecture:

- **Syllabus** — read the full syllabus at <https://aicoding.cs.wisc.edu/fa26/syllabus.html>
- **Pro Git, Chapters 1–3** — <https://git-scm.com/book/en/v2>

Git concepts (clone, branch, commit, push, merge requests) are used immediately in Lecture 1. Come having read these chapters.

Verification Checklist

Run each command and confirm the expected output before the first lecture:

Tool	Command	Expected output
Git	<code>git --version</code>	<code>git version 2.x.x</code> or later
Git identity	<code>git config --global user.email</code>	<code>yournetid@wisc.edu</code>
Node.js	<code>node --version</code>	<code>v20.x.x</code>
npm	<code>npm --version</code>	<code>10.x.x</code> or higher
Python	<code>python3 --version</code>	<code>Python 3.12.x</code>
pip	<code>pip3 --version</code>	<code>pip 24.x</code> or higher
Docker	<code>docker --version</code>	<code>Docker version 25.x</code> or later
Docker Compose	<code>docker compose version</code>	<code>Docker Compose version 2.x</code>

Tool	Command	Expected output
GitLab HTTPS auth	<code>git ls-remote https://git.doit.wisc.edu/.../lectures-<yournetid>.git</code>	Lists refs (branch names/commit hashes), no auth error
Repos cloned	<code>ls lectures-<yournetid>/</code>	Lists the repo contents
Claude Code in repo	Open VS Code in the cloned repo (<code>code lectures-<yournetid></code>), open Claude Code, and ask: "What files are in this project?"	Claude responds with an accurate description of the repo contents

If something fails and you can't resolve it, bring your laptop to the course office hours to get help. The activity for the first lecture will involve actually running these tools, so it's worth resolving anything before then.

Questions? Come to office hours — see the syllabus at aicoding.cs.wisc.edu.